CYBERSHIKSHA CYBERSECURITY CAPSTONE PROJECT REPORT

Network Intrusion Detection System (NIDS) Using Suricata & Grafana Observability Stack

End-to-End Threat Detection, Real-Time EVE JSON Telemetry Parsing, Log Aggregation, and SOC Dashboard Visualization

PROJECT DEVELOPERS
Pragati Gaikwad & Rohit Dhotre

BATCH & PROGRAM IDENTIFICATION
Batch No: 00021 — Cybershiksha

PROJECT MENTOR & GUIDE
Under the Expert Guidance of Rahul Sir

CORE TECHNOLOGY STACK
Suricata 8.0.7, Grafana Alloy, Loki 3.7.0, Grafana, Docker

1. Introduction & Executive Summary

Modern organizational networks face an increasingly hostile threat landscape characterized by automated port scanning, vulnerability enumeration, unauthorized lateral movement, and multi-stage exploitation. Traditional perimeter security controls—such as stateless packet-filtering firewalls—primarily regulate network access based on IP addresses and port numbers, leaving security teams blind to deep packet payloads and internal east-west traffic anomalies.

This project designs, deploys, and validates a production-grade, open-source **Network Intrusion Detection System (NIDS) and Security Operations Center (SOC) Telemetry Pipeline**. Developed by **Pragati Gaikwad** and **Rohit Dhotre** (**Batch No: 00021, Cybershiksha**) under the guidance of **Rahul Sir**, the system integrates **Suricata** as the high-speed packet inspection engine with the cloud-native **Grafana Observability Stack** (**Grafana Alloy**, **Grafana Loki**, and **Grafana**) containerized via Docker. The pipeline inspects live network traffic in real time, serializes structured security alerts into Extensible Event Format (`eve.json`), parses and indexes key network labels, and visualizes live threat telemetry on an interactive SOC dashboard.

2. Problem Statement

- **Lack of Deep Packet Inspection (DPI):** Conventional network firewalls do not analyze application-layer protocol anomalies, suspicious banner grabbing, or stealthy Nmap reconnaissance scans traversing the local subnet.
- **High-Volume Unstructured Log Overload:** While Intrusion Detection Systems generate thousands of raw log entries during an active scan, manually inspecting flat text files via command-line tools causes severe delays in incident triage and threat correlation.
- **Absence of Real-Time Centralized Visualization:** Without an automated log shipping, indexing, and dashboarding pipeline, security analysts cannot monitor alert velocity, track cumulative security incidents, or correlate attacker IP addresses in real time.

3. Motivation

The primary motivation behind this project is to bridge the operational gap between raw packet capture and actionable Security Operations Center (SOC) intelligence. Legacy SIEM solutions often require massive hardware footprints and expensive licensing. By leveraging **Suricata** alongside lightweight, high-performance open-source observability microservices—**Grafana Alloy** for log collection, **Grafana Loki** for label-based indexing, and **Grafana** for visualization—organizations can achieve sub-second threat visibility and automated log analytics with minimal resource overhead.

4. Project Objectives

1. **High-Speed Packet Inspection:** Deploy **Suricata 8.0.7 RELEASE** in system daemon mode using Linux `AF_PACKET` zero-copy packet capture on interface `enp0s3`.
2. **Structured Telemetry Generation:** Configure Suricata outputs to simultaneously record fast line-based alerts (`fast.log`) and comprehensive Extensible Event Format JSON logs (`/var/log/suricata/eve.json`).
3. **Automated Log Parsing & Label Extraction:** Configure **Grafana Alloy** to tail `eve.json`, extract 5-tuple network metadata (`src_ip`, `dest_ip`, `src_port`, `dest_port`, `proto`, `event_type`, `timestamp`), and promote key attributes into indexed Loki labels.
4. **Scalable Log Storage & Querying:** Deploy **Grafana Loki (v3.7.0)** on port `3100` to index incoming log streams and serve high-speed LogQL range queries.
5. **Real-Time SOC Visualization:** Build an interactive **Grafana** dashboard on port `3000` featuring Total Alert counters, Events-per-Minute graphs, Alert Severity Donut charts, and Live IDS Security Alert tables.
6. **Adversary Simulation Validation:** Conduct controlled reconnaissance and vulnerability scans (`ping`, `curl`, `nmap -sV`) from **Kali Linux** against vulnerable targets to verify end-to-end detection accuracy.

5. Proposed System Architecture & Data Flow Pipeline

The proposed architecture operates across a Bridged LAN environment where all virtual machines share a common network segment. Traffic generated between the Kali Linux attacker node and the vulnerable target node is captured by the Ubuntu IDS monitoring server via promiscuous/SPAN interface monitoring and processed through a 6-stage telemetry pipeline.

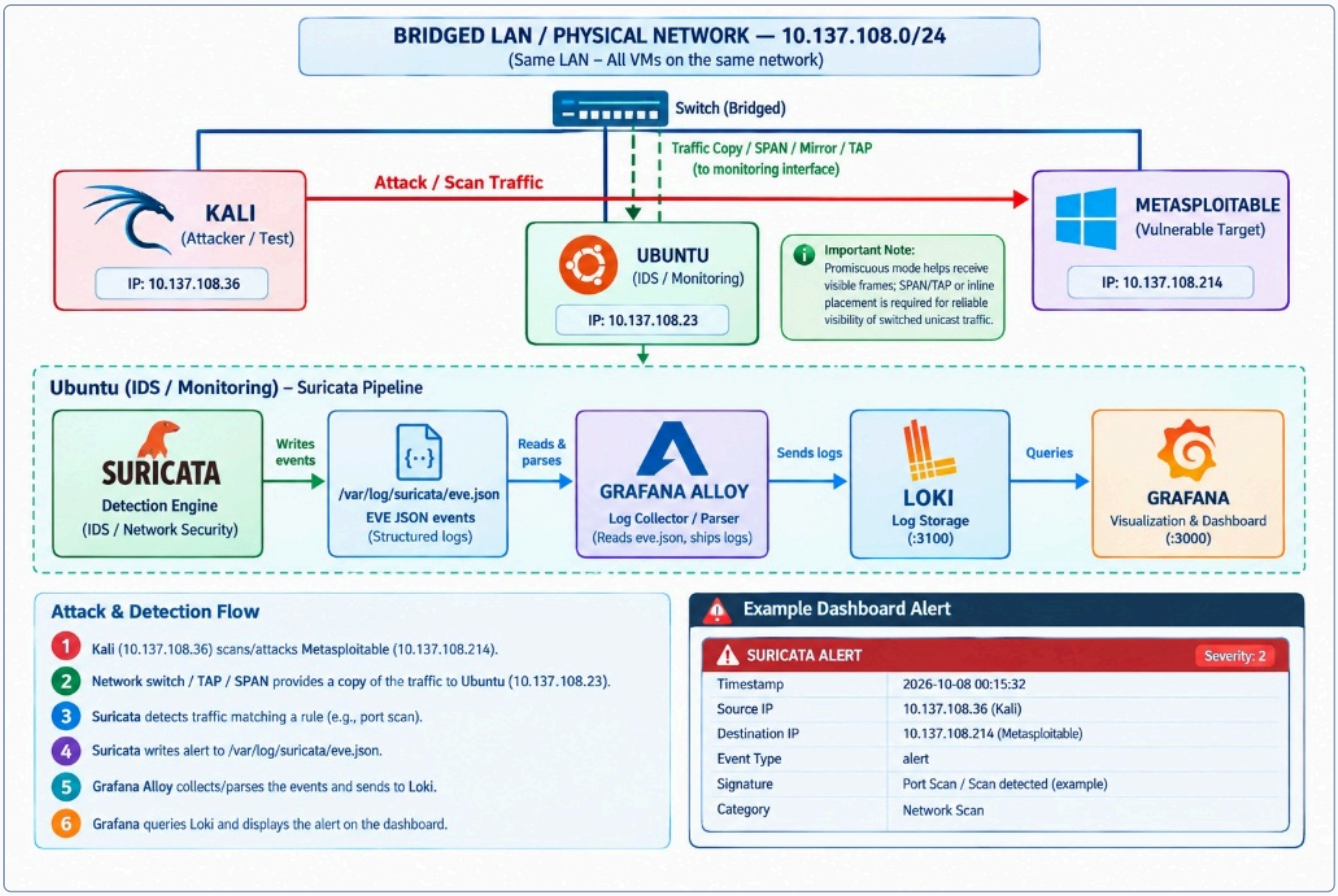


Figure 1: Complete End-to-End Suricata IDS & Grafana SOC Pipeline Architecture Diagram

Illustrating Bridged LAN Topology, Promiscuous/SPAN Traffic Capture, Suricata EVE JSON Logging, Grafana Alloy Parsing, Loki Storage (:3100), and Grafana Visualization (:3000)

5.1 Step-by-Step Telemetry & Detection Workflow

- **Step 1 — Attack & Scan Traffic Generation:** The Kali Linux attacker node initiates ICMP sweeps, HTTP probes, and Nmap service version scans (`nmap -sV`) against the target host.
- **Step 2 — Network Switch / Promiscuous Capture:** The virtual bridged switch mirrors/forwards frames to the Ubuntu IDS server interface (`enp0s3`), which operates in promiscuous mode to capture all visible network traffic.
- **Step 3 — Suricata Deep Packet Inspection:** Suricata's multi-threaded engine inspects packets against loaded signature rules and protocol decoders, detecting reconnaissance scans and protocol anomalies.
- **Step 4 — EVE JSON Event Serialization:** Suricata writes structured JSON alert, anomaly, flow, DNS, and HTTP events directly to `/var/log/suricata/eve.json` .
- **Step 5 — Grafana Alloy Collection & Parsing:** The containerized Grafana Alloy agent reads `/var/log/suricata/eve.json` via a read-only bind mount, parses JSON keys, attaches stream labels

(`job="suricata"` , `event_type` , `proto`), and pushes logs to Loki's HTTP endpoint (`http://loki:3100/loki/api/v1/push`).

- **Step 6 — Grafana Querying & Visualization:** Grafana queries Loki using LogQL expressions (such as `count_over_time({job="suricata"} [5m])`) and updates the live SOC dashboard panels.

6. Hardware, Software & Network Requirements

6.1 Hardware Requirements Specification

Hardware Resource	Minimum Specification	Recommended / Implemented Spec	Operational Role in Project
Processor (CPU)	Dual-Core 2.0 GHz x86_64	Quad-Core 2.5+ GHz with VT-x/AMD-V	Supports Suricata multi-threaded workers (`W: 4 FM: 1 FR: 1`) and concurrent VMs.
Memory (RAM)	4 GB RAM	8 GB – 16 GB System RAM	Allocates memory for Ubuntu IDS server, Docker containers, Kali Linux, and target VM.
Disk Storage	40 GB Free Space	80+ GB SSD Storage	High-throughput I/O for `/var/log/suricata/eve.json` (208+ MB) and Loki index chunks.
Network Interface	100 Mbps Ethernet Adapter	Gigabit Bridged NIC (Promiscuous Mode)	Captures raw Ethernet frames across the bridged subnet via `AF_PACKET` .

6.2 Software Requirements Specification

Software Component	Version / Image	Host / Container Name	Functional Purpose
IDS Host OS	Ubuntu Linux Desktop/Server	`rohit@ids-server`	Hosts native Suricata daemon, `tcpdump` , and Docker Engine runtime.
Attacker OS	Kali GNU/Linux Rolling	`kalipragy@kali`	Executes reconnaissance and port scanning via `ping` , `curl` , and `nmap 7.99` .
IDS Detection Engine	Suricata `8.0.7 RELEASE`	`suricata.service` (PID 1244)	Performs real-time deep packet inspection and outputs `eve.json` and `fast.log` .
Telemetry Collector	`grafana/alloy:latest`	`ids-alloy` (`0e44f91acc19`)	Tails `eve.json` , extracts JSON fields & labels, and ships log streams to Loki.
Log Storage Backend	`grafana/loki:3.7.0`	`ids-loki` (`881375e597c0`)	Indexes and stores log streams; exposes REST query API on `127.0.0.1:3100` .
Visualization UI	`grafana/grafana:latest`	`ids-grafana` (`7cbad6febd1b`)	Renders interactive SOC dashboards on `0.0.0.0:3000` .

6.3 Network Requirements & Topology Details

- **Network Addressing:** Architectural reference subnet `10.137.108.0/24` (Kali: `10.137.108.36`, Ubuntu IDS: `10.137.108.23`, Target: `10.137.108.214`) and active lab runtime subnet `10.242.221.0/24` (Ubuntu IDS/Grafana Host: `10.242.221.225`, Vulnerable Target Host: `10.242.221.145`, Gateway/DNS: `10.242.221.4`).
- **Capture Interface:** Interface `enp0s3` operating in promiscuous mode with Linux `AF_PACKET` zero-copy ring buffer capture.
- **Firewall & Port Bindings:** Port `3000/TCP` exposed across the LAN (`0.0.0.0:3000`) for Grafana UI access; Port `3100/TCP` bound to localhost (`127.0.0.1:3100`) and internal Docker bridge network for secure Loki log ingestion.

7. Detailed Implementation & Execution Evidence

7.1 Phase 1: Adversary Reconnaissance & Vulnerability Scanning (Kali Linux)

From the Kali Linux attacker terminal (`kalipragy@kali`), active network reconnaissance and service enumeration commands were executed to simulate an adversary probing the network infrastructure:

- **ICMP Ping Sweep:** Running `ping -c 4 10.242.221.225` transmitted 4 ICMP Echo Request packets and received 4 replies (`0% packet loss` , `rtt min/avg/max/mdev = 9.195/32.509/76.392/26.816 ms`), verifying connectivity to the monitoring server.
- **HTTP Service Probing:** Probing `curl http://10.242.221.145:3000` returned `Failed to connect... Could not connect to server` (generating TCP SYN/RST connection telemetry), while `curl http://10.242.221.225:3000` returned `<a href="/login">Found</a>.` , confirming the live Grafana web server.
- **Nmap Service & Version Enumeration (`nmap -sV 10.242.221.145`):** Using Nmap 7.99, the scan swept 1,000 TCP ports, identifying 991 closed ports (reset) and **9 open vulnerable services** on target `10.242.221.145` (MAC `1C:CE:51:C8:A3:D7`):
  - `22/tcp open ssh` — OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
  - `80/tcp open http` — Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30)
  - `139/tcp & 445/tcp open netbios-ssn` — Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
  - `143/tcp open imap` — Courier Imapd (released 2008)
  - `443/tcp open ssl/http` — Apache httpd 2.2.14
  - `5001/tcp open java-object` — Java Object Serialization
  - `8080/tcp open http` — Apache Tomcat/Coyote JSP engine 1.1
  - `8081/tcp open http` — Jetty 6.1.25

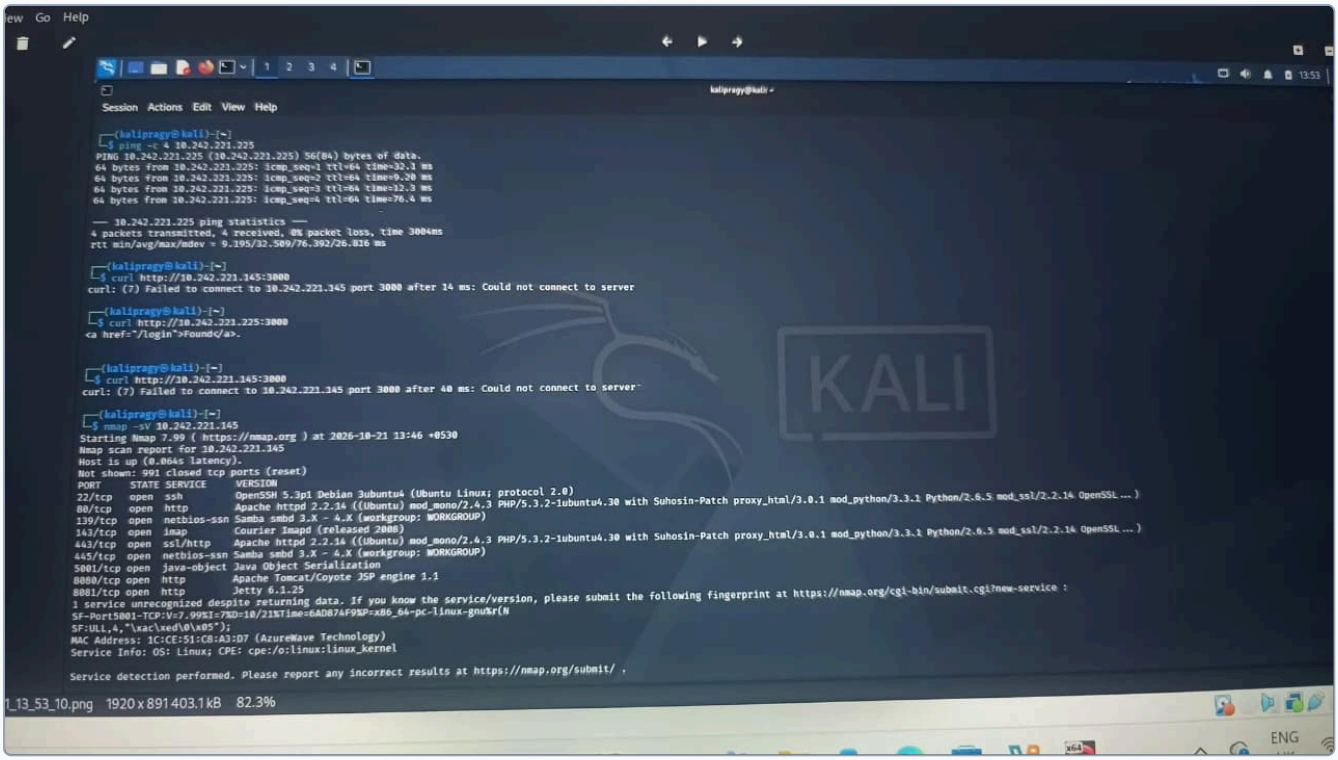


Figure 2: Kali Linux Terminal (kalipragy@kali) — ICMP Ping, HTTP Curl Probing, and Nmap (-sV) Service Enumeration

Demonstrating active reconnaissance and banner grabbing against target 10.242.221.145 and monitoring host 10.242.221.225

7.2 Phase 2: Suricata Engine Configuration (`/etc/suricata/suricata.yaml`)

On the Ubuntu IDS server (`rohit@ids-server`), Suricata's master configuration file (`/etc/suricata/suricata.yaml`) was configured to enable dual logging outputs: `fast.log` for rapid CLI inspection and `eve.json` (Extensible Event Format) for structured JSON serialization.

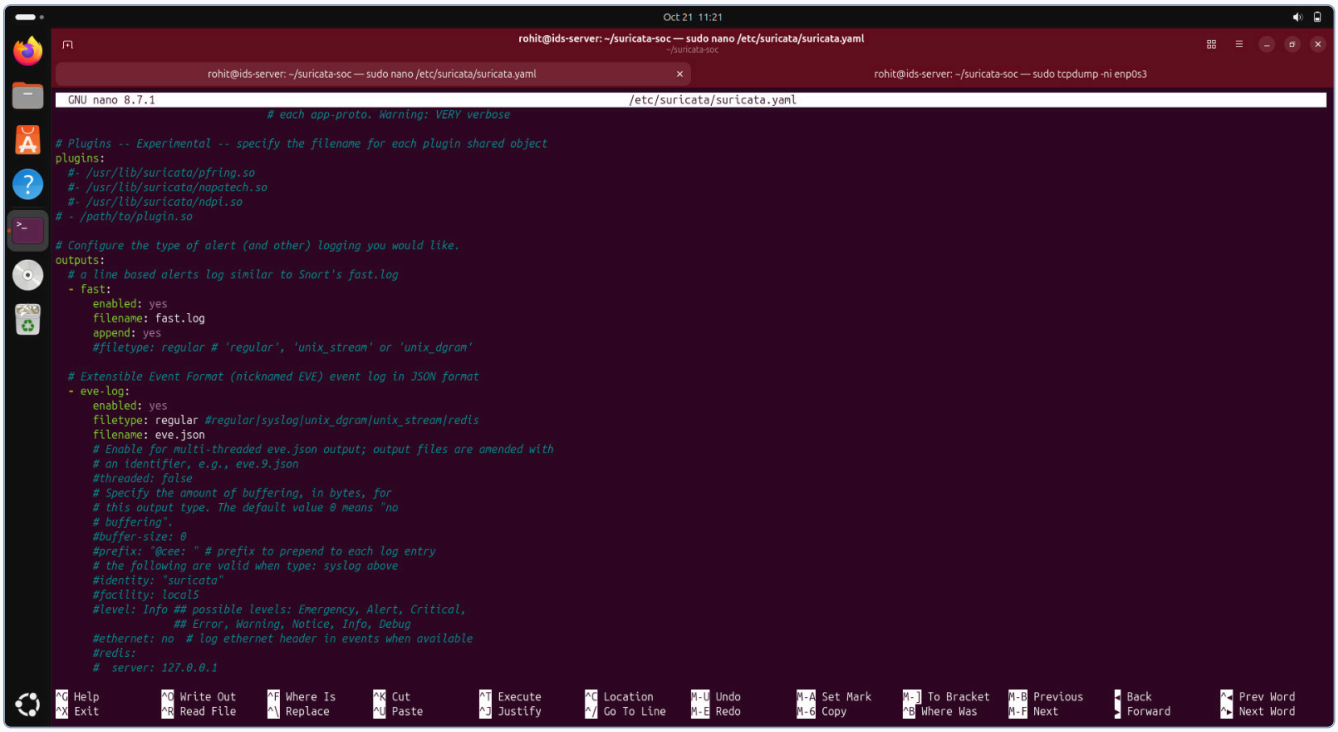


Figure 3: Editing /etc/suricata/suricata.yaml in GNU Nano — Enabling fast.log and eve-log (eve.json) Outputs

Configuring "enabled: yes", "filetype: regular", and "filename: eve.json" under the outputs section

Using `sudo grep -n -A 45 'eve-log:' /etc/suricata/suricata.yaml` , the exact line numbers (lines 98–134) and configuration directives for `eve-log` were verified alongside the running Docker containers (`ids-alloy` , `ids-grafana` , and `ids-loki`):

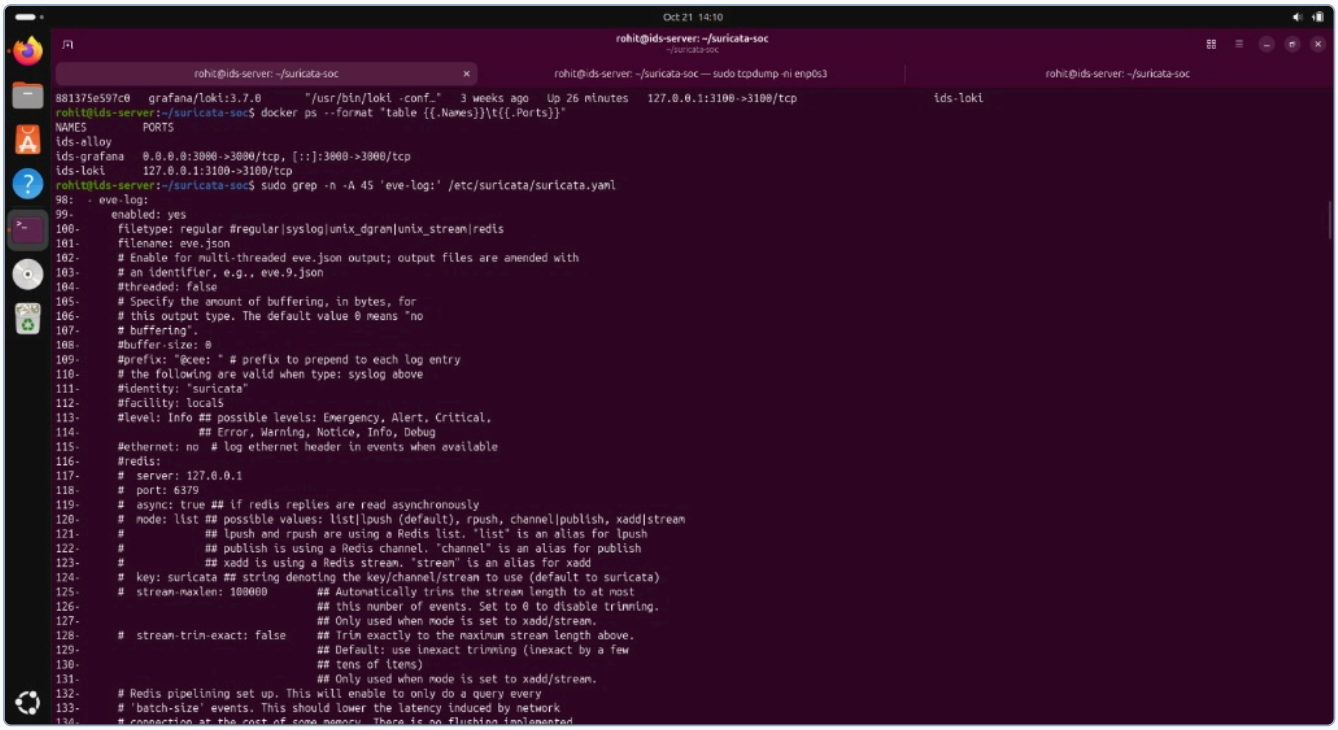


Figure 4: Verifying EVE Log Parameters (Lines 98–134) in /etc/suricata/suricata.yaml & Active Docker Port Mappings

Confirming regular file output to eve.json and verifying container port bindings for Grafana (:3000) and Loki (:3100)

7.3 Phase 3: Suricata Service Validation, Syntax Testing & Container Health Checks

Before launching full telemetry collection, the Suricata configuration syntax was validated using `sudo suricata -T -c /etc/suricata/suricata.yaml`, confirming: `This is Suricata version 8.0.7 RELEASE running in SYSTEM mode. Configuration provided was successfully loaded.`

Furthermore, `sudo systemctl status suricata --no-pager` confirmed the daemon was `active (running)` under Main PID `1244` executing `/usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid --user suricata --group suricata` with 4 worker threads (`W: 4 FM: 1 FR: 1 Engine started`).

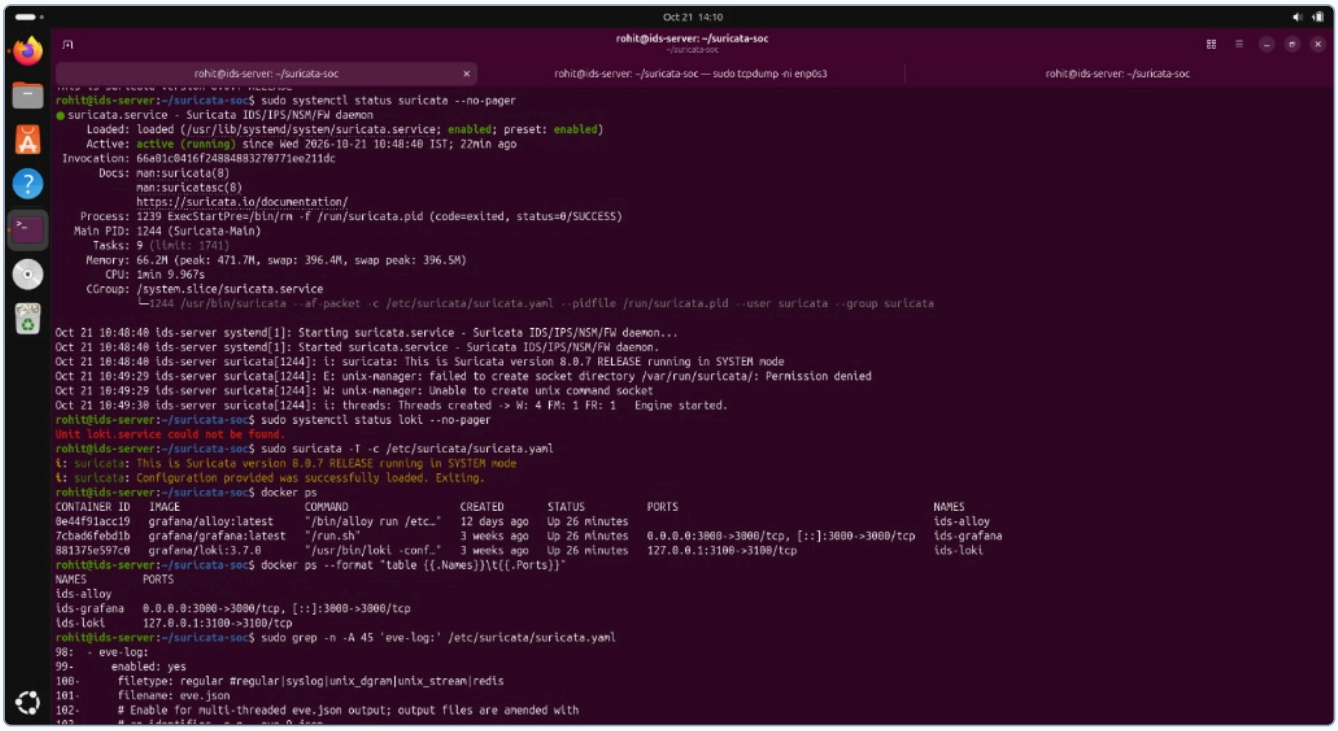


Figure 5: Suricata Daemon Status (Active/Running), Configuration Test (suricata -T), and Docker Container Status

Validating AF_PACKET capture mode, multi-threaded engine startup, and active containers (ids-alloy, ids-grafana, ids-loki)

Next, the readiness of the Grafana Loki log ingester was verified via HTTP health probes (`curl -s http://127.0.0.1:3100/ready`). After completing its 15-second ingester warm-up window, Loki returned `ready`, confirming full operational readiness.

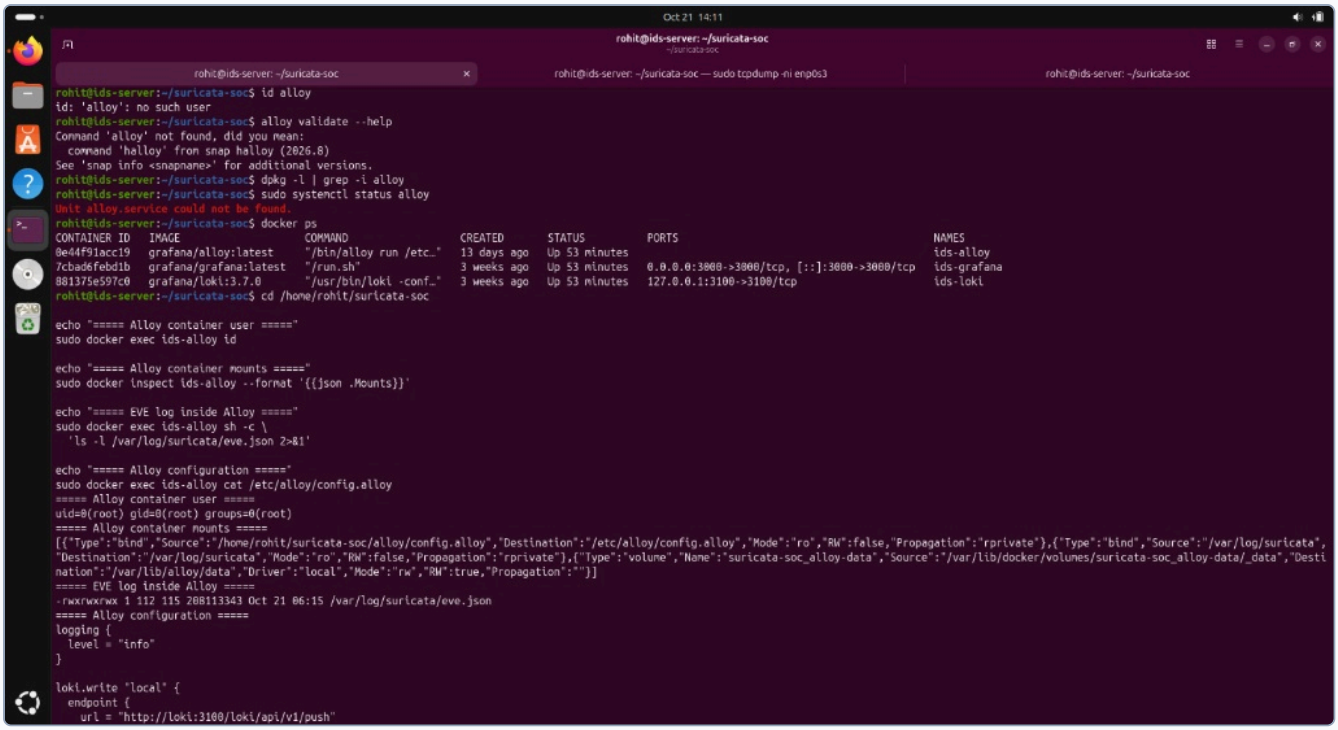


Figure 6: Verifying Continuous Suricata Daemon Uptime & Loki Ingester Readiness Probe (127.0.0.1:3100/ready)

Confirming Loki transitions from "Ingester not ready" to "ready" state for real-time log ingestion

7.4 Phase 4: Grafana Alloy Pipeline Configuration & Container Mount Verification

Because Grafana Alloy runs inside the `ids-alloy` Docker container while Suricata writes logs to the host filesystem at `/var/log/suricata/eve.json`, container diagnostic checks were performed to verify user permissions, volume bind mounts, and file visibility:

- **Container User Check:** `sudo docker exec ids-alloy id` confirmed `uid=0(root) gid=0(root)`, preventing permission-denied errors when reading host logs.
- **Bind Mount Verification:** `sudo docker inspect ids-alloy --format '{{json .Mounts}}'` verified read-only bind mounts for `/home/rohit/suricata-soc/alloy/config.alloy` → `/etc/alloy/config.alloy` and `/var/log/suricata` → `/var/log/suricata`, plus read-write persistence on `/var/lib/alloy/data`.
- **EVE JSON File Size Verification:** Running `ls -l /var/log/suricata/eve.json` inside the container confirmed live access to `208,113,343 bytes` (~208.1 MB) of captured Suricata telemetry.

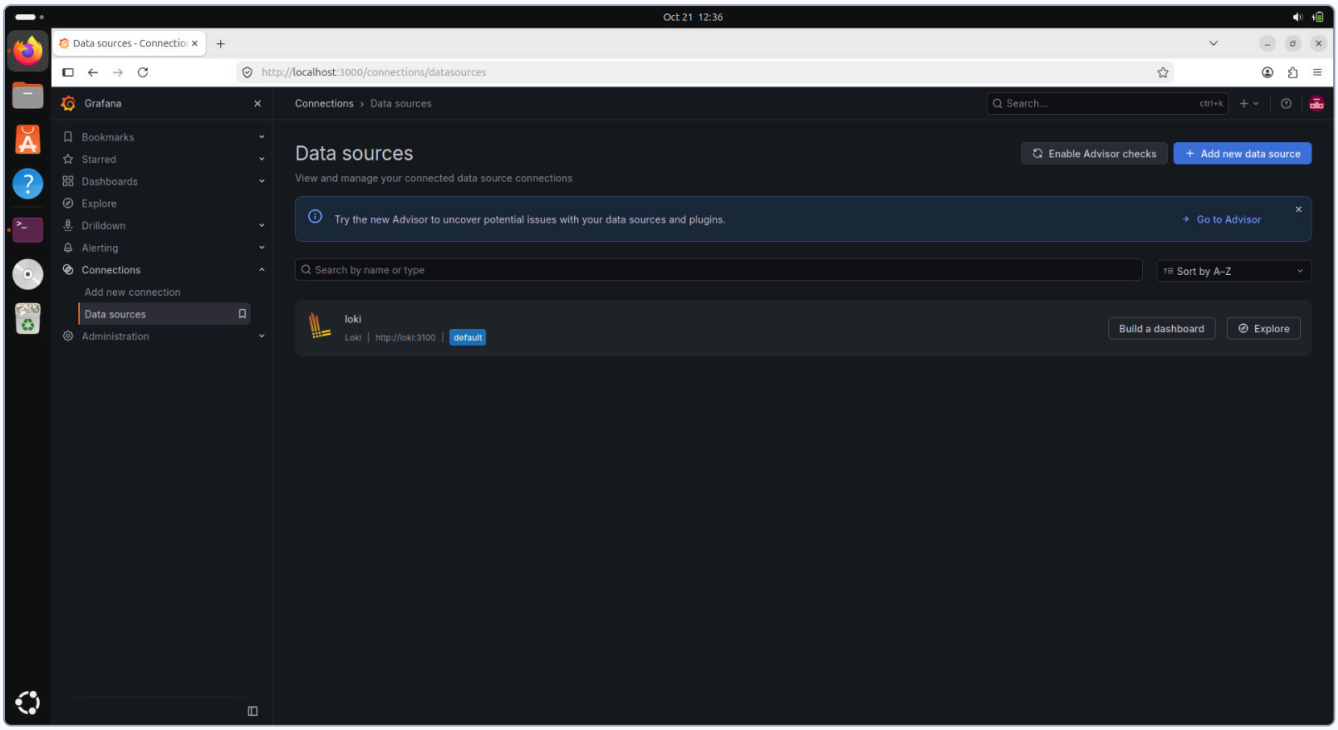


Figure 7: Inspecting ids-alloy Container User Permissions, Volume Mounts, and Live eve.json Access

Verifying root read access, bind mounts, and active 208 MB eve.json file visibility inside the Alloy container

The complete Grafana Alloy configuration (`/home/rohit/suricata-soc/alloy/config.alloy`) defines the file tailing target, JSON extraction expressions, label promotion rules, and Loki push endpoint:

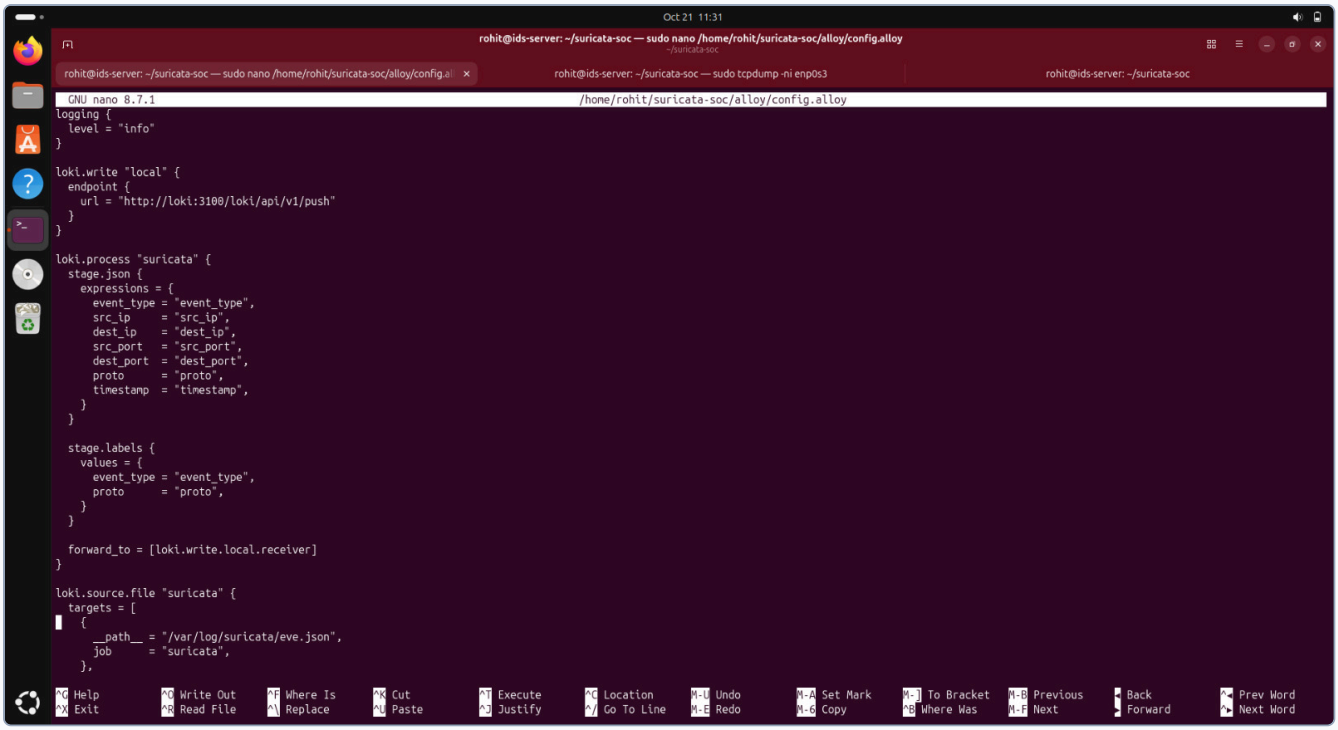


Figure 8: Complete Grafana Alloy Configuration (/home/rohit/suricata-soc/alloy/config.alloy)

Showing loki.write endpoint, loki.process JSON field extraction (event_type, src_ip, dest_ip, proto), stage.labels, and loki.source.file

7.5 Phase 5: Loki API Log Ingestion & Query Range Verification

To confirm that Grafana Alloy was actively parsing and shipping logs into Loki, direct HTTP range queries were executed against Loki's REST API (`http://127.0.0.1:3100/loki/api/v1/query_range`) for `query={job="suricata"}` over the past 1 hour (`since=1h` , `limit=20`). Loki returned `"status":"success"` along with structured JSON log streams containing interface metadata (`"in_iface":"enp0s3"`), source/destination IPs, ports, and protocol labels.

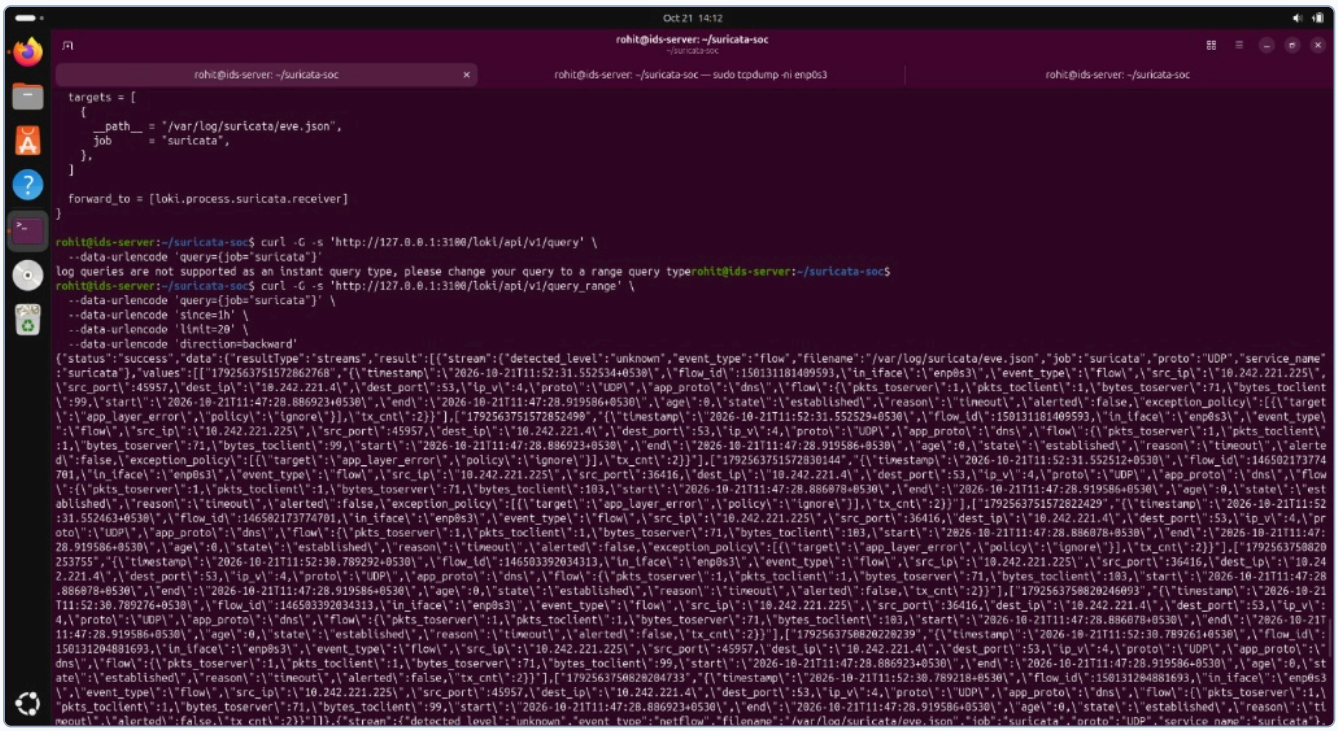


Figure 9: Validating Loki Log Ingestion via HTTP Range Query API (/loki/api/v1/query_range)

Confirming indexed Suricata streams and JSON payload retrieval from Loki port 3100

7.6 Phase 6: Grafana Data Source Connection Setup

Grafana was configured with Grafana Loki as the default data source (`http://loki:3100`) to enable seamless metric querying and log visualization across the dashboard panels.

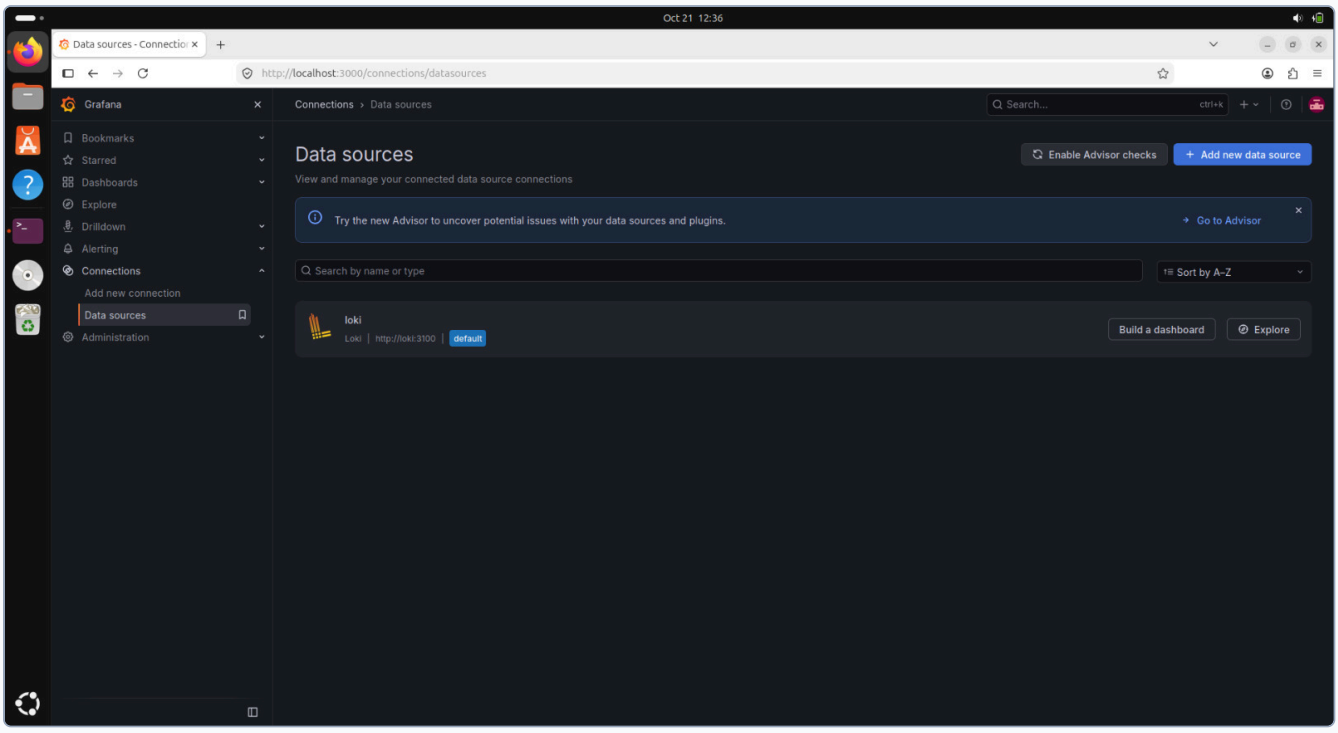


Figure 10: Grafana Data Sources Configuration — Connecting Loki Data Source at http://loki:3100

Verifying active Loki data source connection inside Grafana Connections menu

8. Grafana SOC Dashboard Design & Threat Visualization

With Suricata logs flowing continuously into Loki, custom Security Operations Center (SOC) dashboards were built in Grafana (`http://localhost:3000`). Using the LogQL query `count_over_time({job="suricata"}[5m])` , time-series panels aggregate event frequencies across 5-minute intervals categorized by `event_type` (`alert` , `stats` , `anomaly` , `dhcp`) and transport protocol (`TCP` , `UDP`).

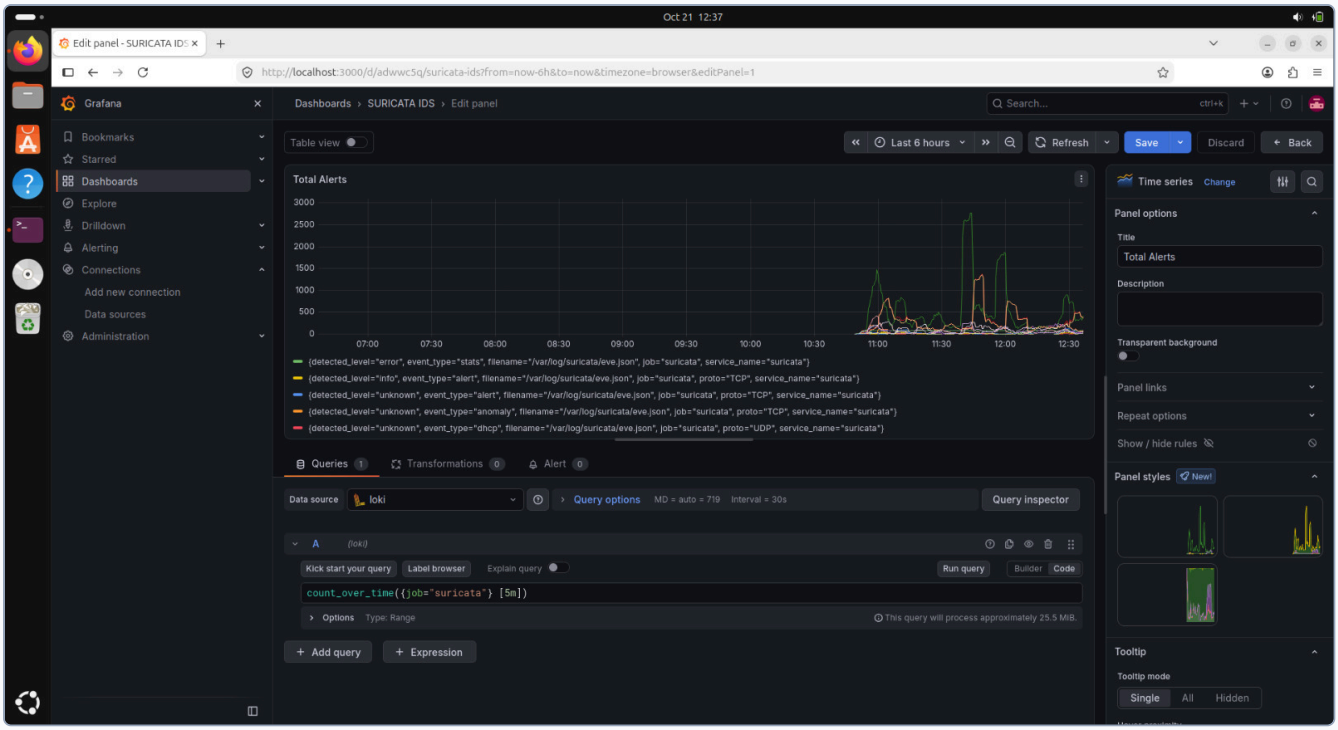


Figure 11: Grafana Panel Editor — Configuring LogQL Query count_over_time({job="suricata"} [5m])

Visualizing multi-series alert and anomaly spikes between 11:00 and 12:30 from the Loki data source

8.1 Complete Security Operations Center (SOC) Executive Views

The finalized **SURICATA IDS Dashboard** provides comprehensive real-time situational awareness across six synchronized panels:

1. **Total Alerts (Time-Series Panel):** Displays real-time event velocity broken down by `event_type` and `proto`, highlighting sharp traffic surges (up to 3,500+ events per window) during active Nmap scans.
2. **Alerts Traffic Over Time:** Isolates high-priority alert bursts across the monitoring window.
3. **Cumulative Bar Chart:** Tracks progressive alert volume growth across time intervals.
4. **Total Security Alerts (KPI Stat Panel):** Displays a prominent counter of **76 Total Security Alerts** captured during the test window.
5. **Events Per Minute (Throughput Graph):** Tracks overall packet inspection volume, showing baseline traffic alongside massive spikes exceeding **2,800 to 6,000+ events/minute** during aggressive port scanning.
6. **Alert Distribution by Severity (Donut Chart) & Live IDS Security Alerts Table:** Visualizes severity proportions alongside a live scrolling log table displaying nanosecond timestamps (`tsNs`), alert actions (`"alert_action": "allowed"`), and triggered Suricata signatures (`Alert: ET SCAN Possible Nmap User-Agent...`) with severity levels.

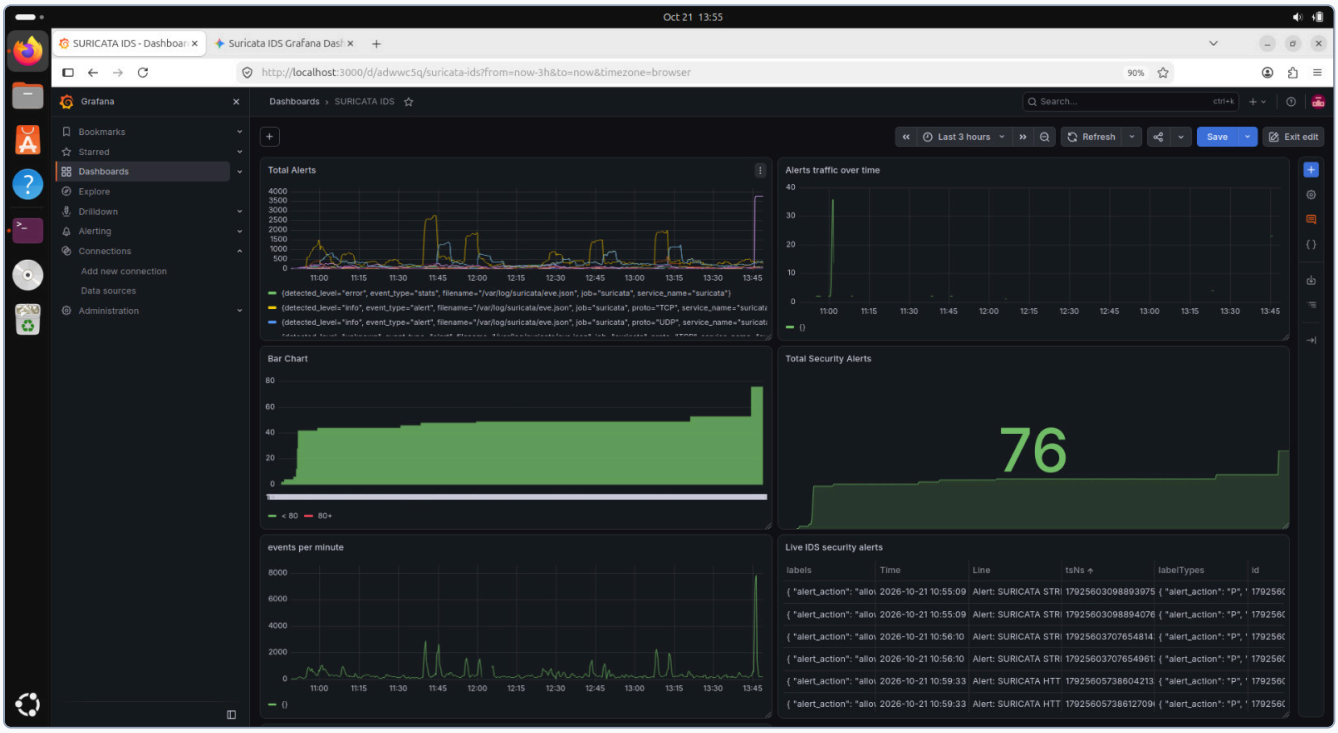


Figure 12: Complete Suricata IDS Dashboard (View 1) — Total Alerts, Alerts Traffic Over Time, Bar Chart, KPI Counter (76), and Live Logs

Real-time SOC monitoring view showing correlated network traffic spikes and 76 triggered security alerts

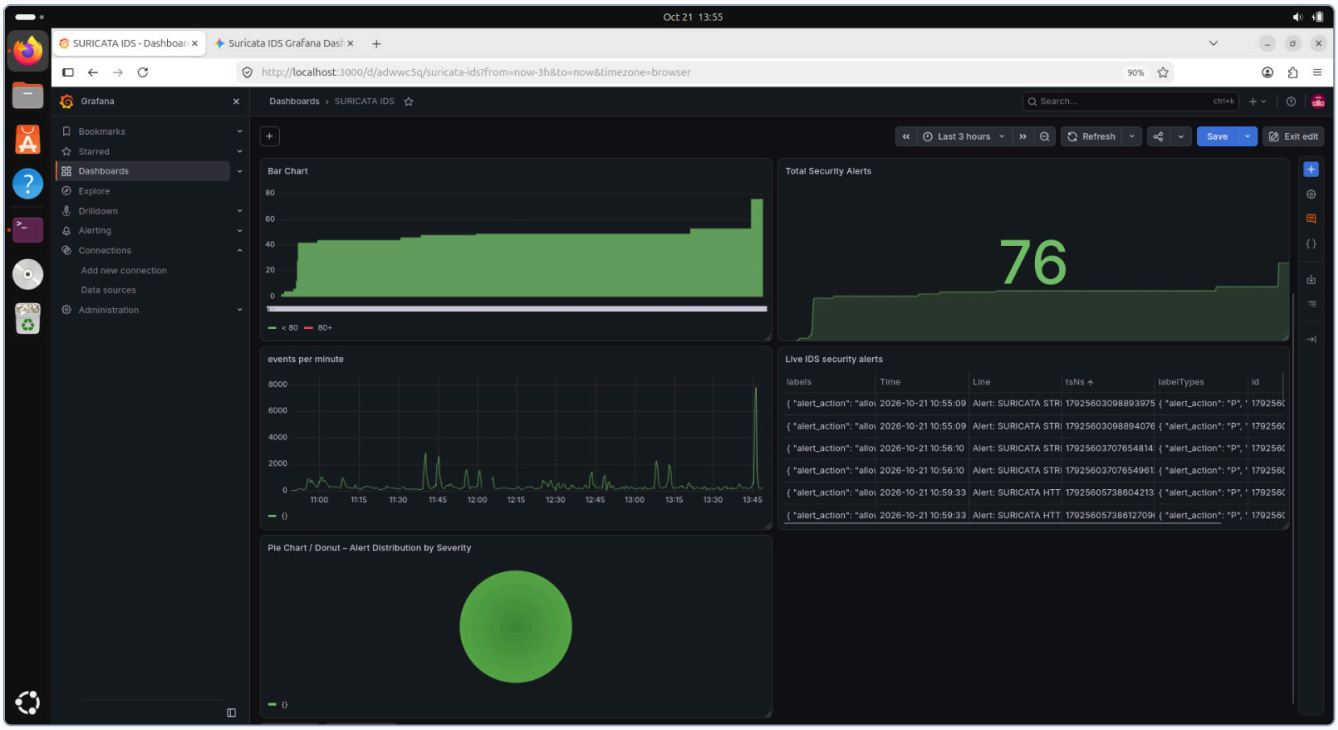


Figure 13: Complete Suricata IDS Dashboard (View 2) — Events Per Minute, Alert Severity Donut Chart & Live IDS Security Alerts Stream

Displaying full-screen SOC analytics including severity distribution donut and real-time signature log table

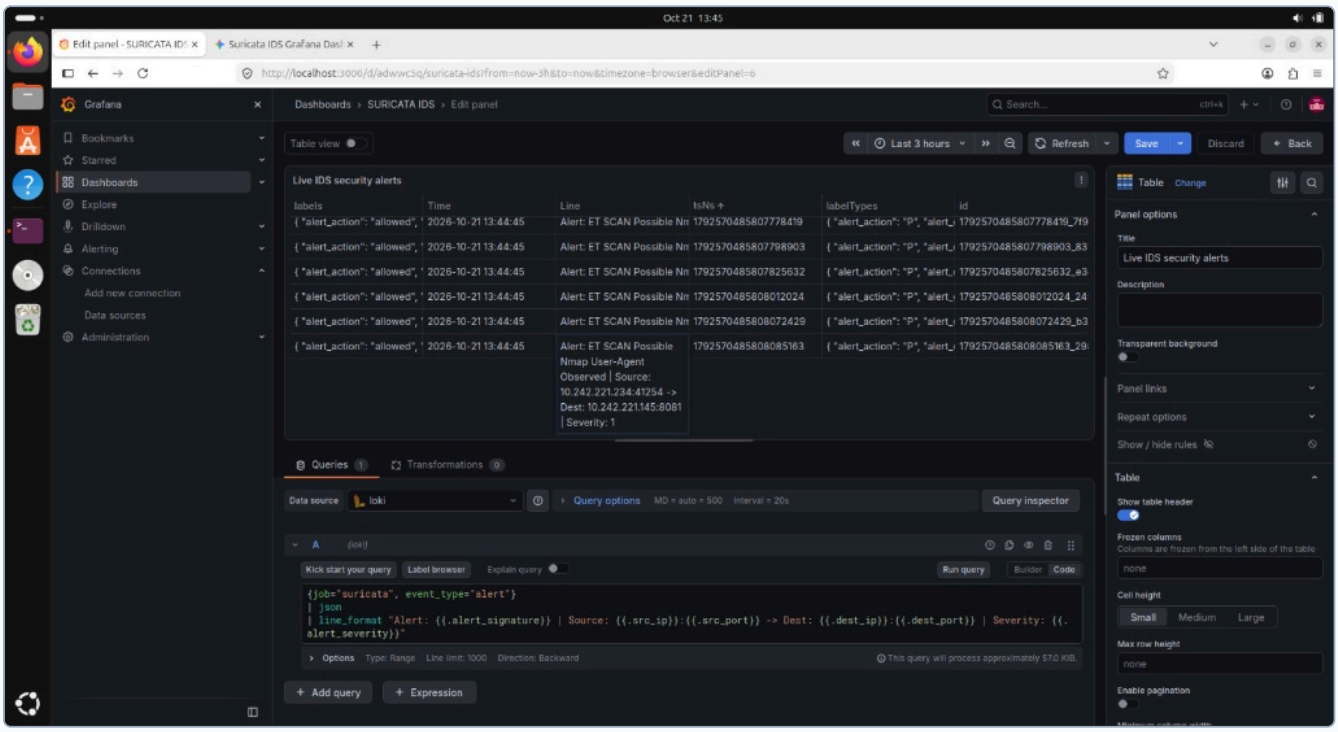


Figure 14: Live IDS Security Alerts Panel Inspection — Triggered Nmap Scan Signatures (Severity 1) & LogQL Query Editor

Inspecting detailed table formatting, json log line formatting, and ET SCAN Nmap User-Agent trigger records (10.242.221.234 -> 10.242.221.145:8081)

9. Conclusion & Future Scope

This project successfully establishes an enterprise-grade, open-source **Network Intrusion Detection System (NIDS)** using **Suricata 8.0.7**, **Grafana Alloy**, **Grafana Loki 3.7.0**, and **Grafana**. Developed by **Pragati Gaikwad** and **Rohit Dhotre** (**Batch No: 00021, Cybershiksha**) under the guidance of **Rahul Sir**, the architecture demonstrated high-speed `AF_PACKET` packet capture, automated `eve.json` parsing, label-based indexing, and real-time threat visualization against live Kali Linux reconnaissance and Nmap enumeration attacks. Future enhancements include transitioning Suricata into inline Intrusion Prevention System (IPS) mode via `NFQUEUE` and configuring automated webhook alerting via Slack/Email.